COMPUTER SCIENCE

O Level Notes

Syllabus 2210 / 0478

Prepared by

Khaled Bin Aziz

01756922708

© 2026 · All Rights Reserved

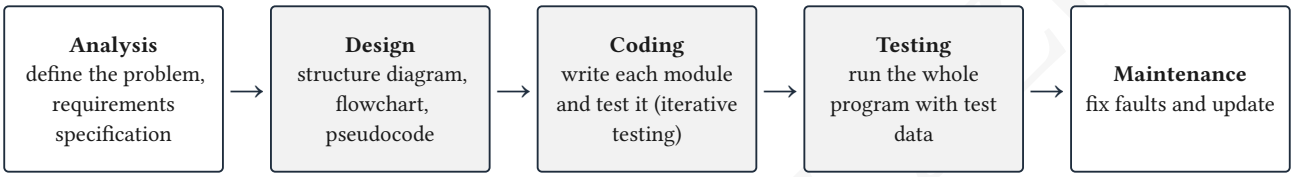
O Level Computer Science

Chapter 7: Algorithm Design and Problem Solving

Life Cycle · Decomposition · Pseudocode · Standard Methods · Validation · Test Data · Trace Tables

7.1 The Program Development Life Cycle

The **program development life cycle** has **five stages**: **analysis, design, coding, testing** and **maintenance**.



The five stages of the program development life cycle

Stage	What happens
Analysis	The problem is **clearly defined** so everyone understands what is needed. The result is the **requirements specification**. It uses **abstraction** and **decomposition**
Design	The specification is used to plan **how** the program will work: **what tasks** are needed, **how each task is done** and **how the tasks work together**. It is documented with **structure diagrams, flowcharts and pseudocode**
Coding and iterative testing	The program is written in a programming language **one module at a time**. Each module is tested, the code is amended and the test is repeated until the module works. This repeating is **iterative testing**
Testing	The **completed** program is run many times with **different sets of test data** to check that **all the parts work together** as the design says
Maintenance	After release the program is kept working: faults are fixed and updates or improvements are made

Abstraction means **keeping the key details** needed for the solution and **removing unnecessary details**. For example, a rail map shows only stations and lines, while a road map shows only roads. The same area needs a **different map for each type of transport**.

Decomposition means **breaking a complex problem into smaller parts**, which can be broken down again until each part is easy to solve. For example, **getting dressed** = select items to wear, remove the clothes being worn, put on the selected items in order.

> **Exam Tip: Iterative testing** happens **during coding** on **individual modules**. The **testing** stage is for the **whole program**. Do not mix them up.

7.2 Computer Systems, Sub-systems and Decomposition

A **computer system** is made up of **software, data, hardware, communications and people**. Every system can be divided into **sub-systems**, and each sub-system can be divided again until each part performs **a single action**. Systems can be very small (an alarm app) or very large (a weather forecast system).

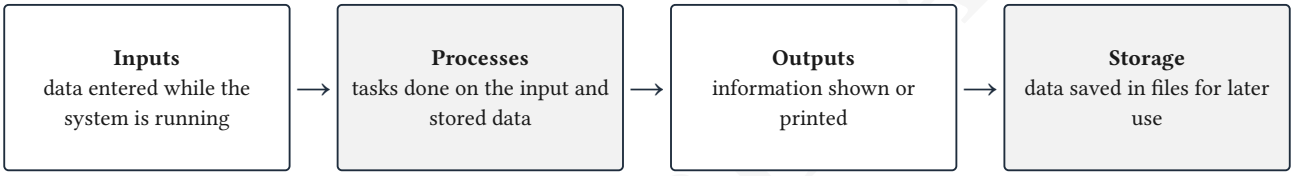
Top-down Design

Top-down design is the decomposition of a system into sub-systems, then breaking each sub-system into smaller sub-systems until each performs just **one action**. The process of breaking down step by step is called **stepwise refinement**. Each sub-system can be written as a **sub-routine**.

Why it is useful:

- Each part is a **smaller, more manageable** problem.
- **Several programmers** can develop and test **different sub-systems at the same time**, which **reduces development and testing time**.
- It is easier to **test** each part separately and to **find errors**.

Decomposing a Problem into Inputs, Processes, Outputs and Storage



The four component parts of any computer system

Part	Alarm app example
Inputs	Time to set the alarm, remove a set alarm, switch the alarm off, press snooze
Processes	Keep checking if the current time matches a set alarm time, store and remove alarm times, manage snooze
Outputs	A continuous sound or tune at the alarm time or when the snooze time ends
Storage	The alarm time(s) that have been set

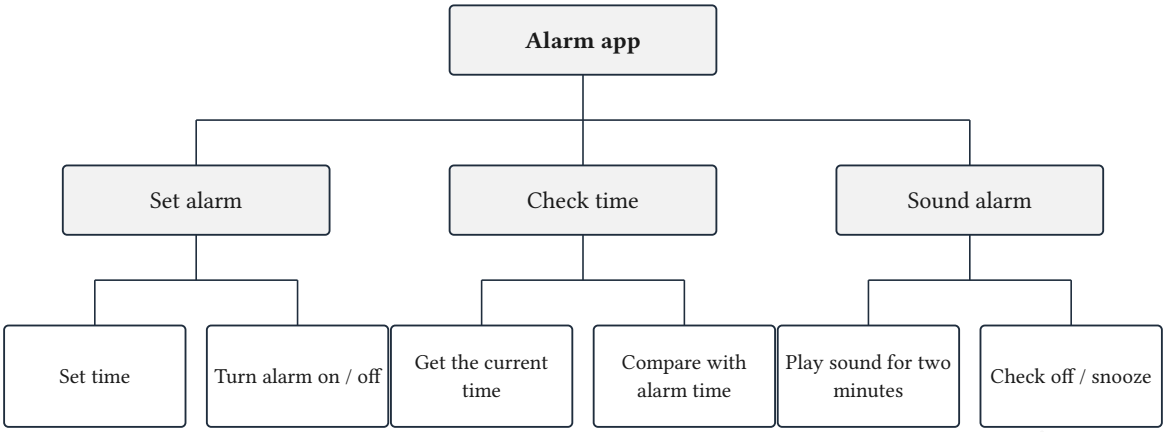
> **Exam Tip:** When asked to **name and describe the component parts** of a system, give all four: **inputs, processes, outputs, storage**, each with a short description and an example from the scenario.

Methods Used to Design a Solution

Three formal methods must be known: **structure diagrams, flowcharts** and **pseudocode**.

Structure diagrams

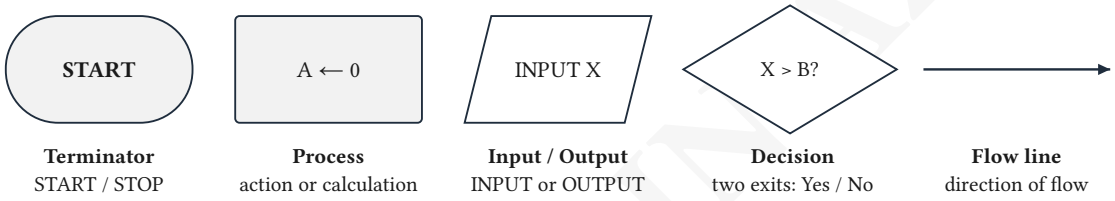
A **structure diagram** is a **hierarchical** diagram that shows top-down design. The top box is the whole system and each lower level gives a **more detailed breakdown** into sub-systems.



Structure diagram (top-down design) for an alarm app

Flowcharts

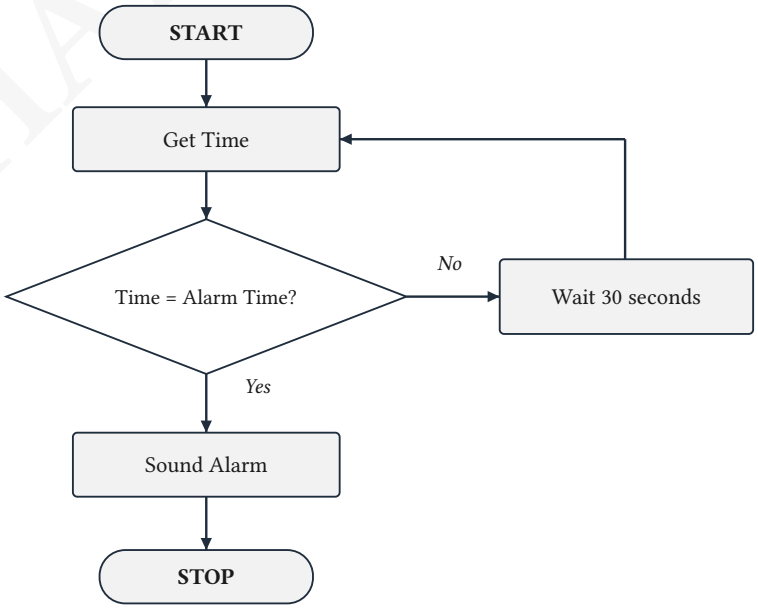
A **flowchart** shows the **steps** of a task and the **order** they are done in. The steps and their order are called an **algorithm**. Flowcharts use standard symbols.



Standard flowchart symbols

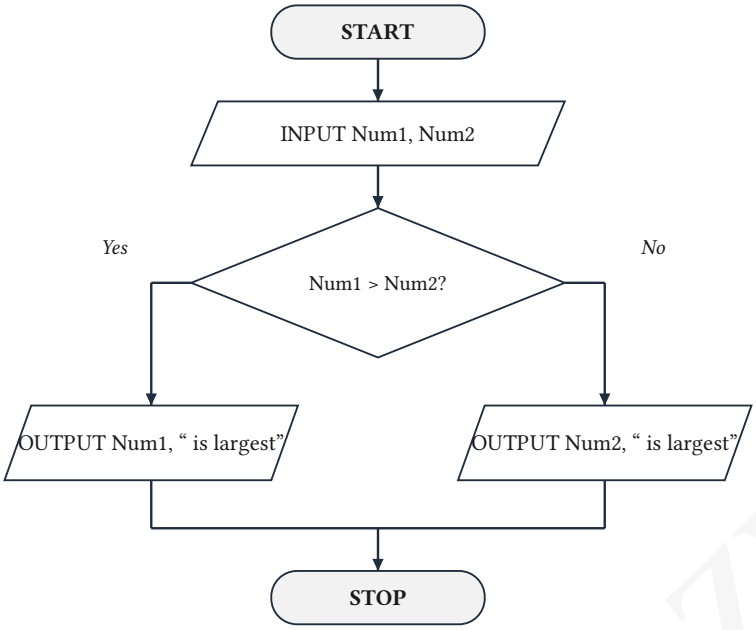
Rules for drawing flowcharts:

- Start with **START** and end with **STOP**.
- A **decision** symbol always has **exactly two exits**, and **both are labelled** (Yes / No).
- Flow lines have **arrows** and usually go top to bottom and left to right.
- A process defined elsewhere (a sub-routine) is shown by its name in the process symbol.



Flowchart: the "check time" sub-system of the alarm app (a loop)

KHALED BIN AZIZ



Flowchart: output the larger of two numbers (selection)

Pseudocode

Pseudocode describes an algorithm using **English keywords similar to a high-level programming language**, but it does **not** follow strict syntax rules. The syllabus style is:

- **Keywords in capital letters** (INPUT, OUTPUT, IF, WHILE …).
- **Names of variables and sub-routines start with a capital letter** and are **meaningful** (for example `StudentMark`, not `s`).
- Statements inside an IF or a loop are **indented by two spaces**.
- A **non-proportional (monospaced) font** is used.

Assignment. The arrow ← gives a variable the value of the expression on its right.

```
Cost ← 10
Price ← Cost * 2          // Price is 20
Tax ← Price * 0.12        // Tax is 2.4
SellingPrice ← Price + Tax // 22.4
Gender ← "M"
Chosen ← False
```

Operator	Action	Operator	Comparison
+	Add	>	Greater than
-	Subtract	<	Less than
*	Multiply	=	Equal to
/	Divide	>=	Greater than or equal to
^	Raise to the power	<=	Less than or equal to
( )	Group (brackets first)	<>	Not equal to
		AND OR NOT	Combine conditions: both / either / not

Input and output. `INPUT` stores data entered in a variable. `OUTPUT` shows a message or value on the screen. Several items are separated by commas. (`READ` is used for files and `PRINT` for a printed copy.)

```
OUTPUT "Enter your name "
INPUT Name
OUTPUT "Your name is ", Name
```

Selection (conditional statements)

IF … THEN … ELSE … ENDIF. The THEN part runs if the condition is true and the ELSE part runs if it is false (ELSE is optional).

```
IF Age < 18
  THEN
    OUTPUT "Child"
  ELSE
    OUTPUT "Adult"
ENDIF
```

A **nested IF** is an IF inside another IF. Here the mark is only tested for pass or fail if it is in the valid range:

```
IF PercentageMark < 0 OR PercentageMark > 100
  THEN
    OUTPUT "Invalid Mark"
  ELSE
    IF PercentageMark > 49
      THEN
        OUTPUT "Pass"
      ELSE
        OUTPUT "Fail"
    ENDIF
ENDIF
```

CASE OF … OTHERWISE … ENDCASE. Used to choose between **several values** of one variable. OTHERWISE is used for every other value.

```
CASE OF Choice
  1 : Answer ← Num1 + Num2
  2 : Answer ← Num1 - Num2
  3 : Answer ← Num1 * Num2
  4 : Answer ← Num1 / Num2
  OTHERWISE OUTPUT "Please enter a valid choice"
ENDCASE
```

Iteration (loops)

	`FOR ... TO ... NEXT`	`REPEAT ... UNTIL`	`WHILE ... DO ... ENDWHILE`
Used when	The **number of repeats is known**	The number of repeats is **not known**	The number of repeats is **not known**
Condition tested	Counter is managed **automatically**	**At the end** (post-condition). Loop stops when the condition becomes **TRUE**	**At the start** (pre-condition). Loop continues while the condition is **TRUE**
Runs at least once?	Yes (if the range is valid)	**Yes, always at least once**	**No**, it may never run

All three can display ten stars:

```
FOR Counter ← 1 TO 10
  OUTPUT "*"
NEXT Counter
```

```
Counter ← 0
REPEAT
  OUTPUT "*"
  Counter ← Counter + 1
UNTIL Counter > 9
```

```
Counter ← 0
WHILE Counter < 10 DO
  OUTPUT "*"
  Counter ← Counter + 1
ENDWHILE
```

Reading ten names into an array with a FOR loop:

```
FOR Counter ← 1 TO 10
  OUTPUT "Enter name of student "
  INPUT StudentName[Counter]
NEXT Counter
```

REPEAT loop that stops when –1 is entered (the first mark is processed before the test), and the same task with **WHILE** (the first value is read **before** the loop and the next one is read **at the end** of the loop):

```
Total ← 0
REPEAT
  INPUT Mark
  IF Mark <> -1
    THEN Total ← Total + Mark
  ENDIF
UNTIL Mark = -1
```

```
Total ← 0
INPUT Mark
WHILE Mark <> -1 DO
  Total ← Total + Mark
  INPUT Mark
ENDWHILE
```

> **Exam Tip:** Choose the loop and **explain why**: **FOR** when the number of repeats is known (for example 10 numbers), **WHILE** when the loop might need to run zero times, **REPEAT … UNTIL** when it must run at least once (for example input validation). Remember **UNTIL stops when TRUE** and **WHILE continues while TRUE**.

7.3 Explaining the Purpose of an Algorithm

An algorithm is an **ordered set of steps to solve a problem**. Its **purpose** is **what it achieves** (not a line-by-line description). Its **processes** are the actions it carries out (inputs, calculations, comparisons, loops, outputs).

Example:

```
REPEAT
  Wait (10)
  Get (Time)
UNTIL Time = Alarm_Time
OUTPUT AlarmSound
```

Purpose: to output the alarm sound at the correct time. **Processes:** wait 10 seconds, get the current time from the system clock, compare it with the alarm time (repeat until they match), output the alarm sound.

Example (the flowchart on the previous pages): it inputs two numbers, compares them and outputs the larger. Input 7 and 18 gives the output `18 is largest`.

How to find the purpose when variables are single letters (A, B, C, X):

1. Look at the **start values** (what is each variable set to?).
2. Notice what is **input** and how often the **loop** repeats.
3. Do a **quick dry run** with small test data.
4. State the purpose in **one sentence** about the **output** (for example "finds the largest and smallest of ten numbers").

7.4 Standard Methods of Solution

Totalling

Keep a **running total** that each value is added to. **Always set the total to 0 first (initialise).**

```
Total ← 0
FOR Counter ← 1 TO ClassSize
  Total ← Total + StudentMark[Counter]
NEXT Counter
```

Counting

Keep a **count** of how many times something happens. **Initialise the count to 0**, then add 1 each time.

```
PassCount ← 0
FOR Counter ← 1 TO ClassSize
  INPUT StudentMark
  IF StudentMark >= 50
    THEN PassCount ← PassCount + 1
  ENDIF
NEXT Counter
```

Counting can also **count down**, for example stock in a shop:

```
NumberInStock ← NumberInStock - 1
IF NumberInStock < 20
  THEN CALL Reorder()
ENDIF
```

> **Exam Tip:** **Totalling** adds **the value** (`Total ← Total + Mark`). **Counting** adds **one** (`Count ← Count + 1`). Examiners often check that you know the difference.

Maximum, Minimum and Average

Method 1: use known limits. Set the maximum to the lowest possible value and the minimum to the highest possible value.

```
MaximumMark ← 0
MinimumMark ← 100
FOR Counter ← 1 TO ClassSize
  IF StudentMark[Counter] > MaximumMark
    THEN MaximumMark ← StudentMark[Counter]
  ENDIF
  IF StudentMark[Counter] < MinimumMark
    THEN MinimumMark ← StudentMark[Counter]
  ENDIF
NEXT Counter
```

Method 2 (better): use the first item. This works for **any** set of values. The loop then starts at the **second** item.

```
MaximumMark ← StudentMark[1]
MinimumMark ← StudentMark[1]
FOR Counter ← 2 TO ClassSize
  IF StudentMark[Counter] > MaximumMark
    THEN MaximumMark ← StudentMark[Counter]
  ENDIF
  IF StudentMark[Counter] < MinimumMark
    THEN MinimumMark ← StudentMark[Counter]
  ENDIF
NEXT Counter
```

Average (mean) = total of the values divided by how many values there are. It is **totalling**, then dividing **after the loop**.

```
Total ← 0
FOR Counter ← 1 TO ClassSize
  Total ← Total + StudentMark[Counter]
NEXT Counter
Average ← Total / ClassSize
```

Linear Search

A **linear search** checks **each item in the list in turn** from the start until the value is found or the end of the list is reached. A **flag** (`Found`) records whether it has been found.

```
OUTPUT "Please enter name to find "
INPUT Name
Found ← FALSE
Counter ← 1
REPEAT
  IF Name = StudentName[Counter]
    THEN
      Found ← TRUE
    ELSE
      Counter ← Counter + 1
  ENDIF
UNTIL Found OR Counter > ClassSize
IF Found
  THEN
    OUTPUT Name, " found at position ", Counter
  ELSE
    OUTPUT Name, " not found."
ENDIF
```

The loop stops when the item is **found** or the **whole list has been searched**. If the same value can appear many times, search the **whole list** and **count** the matches (for example, how many chose “ice cream”):

```
ChoiceCount ← 0
FOR Counter ← 1 TO Length
  IF Dessert[Counter] = "ice cream"
    THEN ChoiceCount ← ChoiceCount + 1
  ENDIF
NEXT Counter
OUTPUT ChoiceCount, " chose ice cream"
```

Bubble Sort

A **bubble sort** puts a list in order (for example ascending). **Compare each item with the next one and swap them if they are in the wrong order.** After one **pass** the **largest** item has “bubbled” to the end and is in its correct place, so the next pass can stop one place earlier. The passes repeat until **no swaps are made** in a pass (or only one item is left).

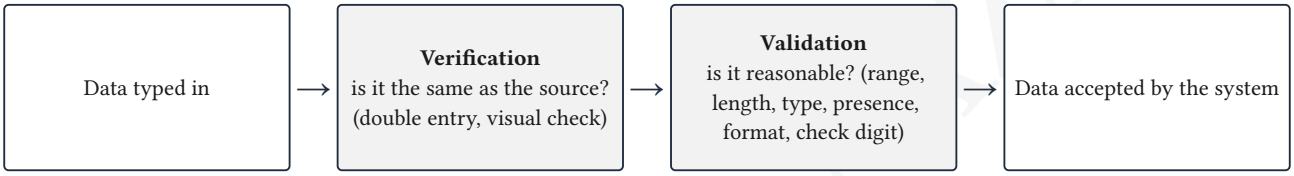
```
First ← 1
Last ← 10
REPEAT
  Swap ← FALSE
  FOR Index ← First TO Last - 1
    IF Temperature[Index] > Temperature[Index + 1]
      THEN
        Temp ← Temperature[Index]
        Temperature[Index] ← Temperature[Index + 1]
        Temperature[Index + 1] ← Temp
        Swap ← TRUE
    ENDIF
  NEXT Index
  Last ← Last - 1
UNTIL (NOT Swap) OR Last = 1
```

Worked example: sort 6, 2, 9, 4 into ascending order.

Stage	1	2	3	4	What happened
Start	6	2	9	4	Unsorted list
Pass 1	2	6	4	**9**	6,2 swapped; 6,9 fine; 9,4 swapped. 9 is in place
Pass 2	2	4	**6**	9	2,6 fine; 6,4 swapped. 6 is in place
Pass 3	2	4	6	9	No swaps, so the list is sorted and the loop stops

> **Exam Tip:** The `Temp` variable is **needed to swap**: copy one value into `Temp` first, otherwise one value is overwritten and lost. The `Swap` **flag** lets the sort stop early when the list is already in order.

7.5 Validation and Verification



Both checks happen when data is entered (they do different jobs)

Validation

Validation is the **automated check by a program that data is reasonable** before it is accepted. If data is rejected, the program should output a **message explaining why** and give **another chance** to enter it. More than one check can be used on the same data (for example a mark can have a range, type and presence check).

Check	What it checks	Example
Range check	A number is **between an upper and a lower value**	Percentage mark between 0 and 100 inclusive
Length check	Data has an **exact** number of characters, or a number of characters **between limits**	Password exactly 8 characters; family name between 2 and 30 characters
Type check	Data is the **correct data type**	Number of brothers must be an integer (whole number)
Presence check	Something has been **entered** (not left blank)	An email address must be filled in
Format check	Characters follow a **set pattern**	Part number XXX999: three letters then three digits
Check digit	The **last digit** of a code is calculated from the other digits and is recalculated when the code is entered	ISBN, barcodes, product codes, VIN

Range check

```
OUTPUT "Please enter the student's mark "
REPEAT
  INPUT StudentMark
  IF StudentMark < 0 OR StudentMark > 100
    THEN
      OUTPUT "Mark must be 0 to 100, please re-enter"
  ENDIF
UNTIL StudentMark >= 0 AND StudentMark <= 100
```

Length check (`LENGTH` returns the number of characters in a string)

```
REPEAT
  INPUT Password
  IF LENGTH(Password) <> 8
    THEN
      OUTPUT "Password must be exactly 8 characters"
  ENDIF
UNTIL LENGTH(Password) = 8
```

Type check (an integer is unchanged by `DIV(x, 1)`, which gives the whole-number part)

```
REPEAT
  INPUT NumberOfBrothers
  IF NumberOfBrothers <> DIV(NumberOfBrothers, 1)
    THEN
      OUTPUT "This must be a whole number, please re-enter"
  ENDIF
UNTIL NumberOfBrothers = DIV(NumberOfBrothers, 1)
```

Presence check

```
REPEAT
  INPUT EmailAddress
  IF EmailAddress = ""
    THEN
      OUTPUT "Required: please enter your email address"
  ENDIF
UNTIL EmailAddress <> ""
```

Check digits catch mistakes made when typing or scanning:
- an **incorrect digit** (5327 instead of 5307),
- a **transposition** of two digits (5037 instead of 5307),
- an **omitted or extra digit** (537 or 53107 instead of 5307),
- **phonetic errors** (13 and 30 sound alike).

A check digit is a **data entry check**, not a data transmission check.

> **Exam Tip: Validation does not prove that data is correct**, only that it is **reasonable**. For example, an age typed as 17 instead of 71 passes a range check of 5 to 100.

Verification

Verification checks that data has been **accurately copied** from one source to another (for example from a paper form into the computer). Two methods:

Method	How it works
Double entry	The data is **typed in twice** (sometimes by **different operators**). The computer **compares both entries** and, if they differ, outputs an **error message** and asks for the data again. Used for passwords and email addresses
Screen / visual check	After entry the data is **displayed** and the **user checks it** against the source document (or from their own knowledge) and **confirms** it is correct before continuing

(Parity checks and checksums verify data **during transfer** and are covered in Chapter 2.)

Validation compared with Verification

Validation	Verification
Checks that data is **reasonable / sensible**	Checks that data has been **copied accurately** (has not changed)
Done **automatically by the program**	Done by **double entry** or by the **user** checking the screen
Can reject data that is **impossible**	Can catch a typing mistake even if the wrong value looks **reasonable**
Example: range check on a mark	Example: typing a password twice

Applying the Checks (typical exam answer)

Data item	Validation checks (with reason)	Verification
Name	**Presence** (cannot be blank), **length** (not too short or too long), **type** (letters only, no digits)	Visual check
Date of birth	**Format** (dd/mm/yyyy), **range** (a realistic date, not in the future), **presence**	Visual check
Password	**Length** (minimum number of characters), **presence**, **format** (must include a digit)	**Double entry**
Phone number	**Type** (digits only), **length** (correct number of digits), **presence**, **format**	**Double entry** or visual check

> **Exam Tip:** Always give **the name of the check and the reason it fits that data**. "Length check, so that the phone number has exactly the right number of digits" earns the mark; "a check" does not.

7.6 Test Data

A **set of test data** is all the items of data needed to work through a solution. A solution is tested to prove it **does what it should** and **does not do what it should not**. Each sub-system is usually tested separately before the whole system. For each test the **expected result** is written down **before** testing and is compared with the **actual result**.

Type	Meaning	Example: mark 0 to 100, whole numbers
Normal	Data the program is **expected to accept**	50, 72, 18 (accepted)

Abnormal (erroneous)	Data that **must be rejected**	−12, 150, "eleven" (all rejected)
Extreme	The **largest and smallest values that are still accepted**	0 and 100 (accepted)
Boundary	The **edge between accepted and rejected data**: **two values at each boundary**, one accepted and one rejected	Lower boundary: −1 (rejected), 0 (accepted). Upper boundary: 100 (accepted), 101 (rejected)

> **Exam Tip:** **Extreme** data is the **largest/smallest accepted** value only. **Boundary** data is the extreme value **plus the nearest rejected value**. Abnormal data includes the **wrong data type** (letters) as well as numbers that are out of range.

Example: the concert ticket algorithm (tickets cost $20 each, 10% discount for 10 or more, 20% discount for 20 or more, maximum 25 per transaction):

Test data	Type	Expected result
0 and 26	Boundary (rejected)	Both rejected
1 and 25	Extreme (accepted)	Cost 20 and cost 400
9 and 10	Boundary of the discount	Cost 180 and cost 180 (10 tickets with 10% off)
19 and 20	Boundary of the discount	Cost 342 and cost 320
−5, "ten"	Abnormal	Rejected
15	Normal	Cost 270

7.7 Trace Tables and Dry Runs

A **dry run** is working through an algorithm **by hand**, step by step, with test data. A **trace table** records the result of the dry run. It has **one column for each variable** and **one for the output**. A new value is written **every time a variable changes**, and every value that is output goes in the output column.

Example: find the average of three numbers. Test data: 5, 3, 8.

```
Total ← 0
Count ← 0
REPEAT
  INPUT Number
  Total ← Total + Number
  Count ← Count + 1
UNTIL Count = 3
Average ← Total / Count
OUTPUT Average
```

Total	Count	Number	Average	OUTPUT
0	0			
5	1	5		

8	2	3		
16	3	8		
			5.33	5.33

Tips for trace tables:
- Put the **starting values** (for example `Total ← 0`) in the **first row**.
- Only write a value when it **changes**. Do not copy unchanged values down.
- Do not put quotation marks around text in the OUTPUT column. A prompt such as `OUTPUT "Enter ten values"` is the **first output** if it comes first.
- Follow the **condition** of the loop exactly to decide when to stop.

Example from the textbook: the algorithm below uses `A` as a counter, `B` as the biggest and `C` as the smallest.

```
A ← 0
B ← 0
C ← 100
REPEAT
  INPUT X
  IF X > B
    THEN B ← X
  ENDIF
  IF X < C
    THEN C ← X
  ENDIF
  A ← A + 1
UNTIL A = 10
OUTPUT B, C
```

Purpose: it inputs ten numbers and outputs the **largest (B) and smallest (C)**. Using the test data 9, 7, 3, 12, 6, 4, 15, 2, 8, 5, the output is `15  2`.

7.8 Identifying and Correcting Errors

Trace tables with test data show **where an algorithm goes wrong**. Test the example above with the data 400, 800, 190, 170, 300, 110, 600, 150, 130, 900. The output is `900  100`, but the smallest number is **110**, so there is an **error**: the start values `B ← 0` and `C ← 100` only work for numbers from 0 to 100. Negative numbers and numbers above 100 break it.

Improving it:
- **Better:** set `B ← -1000000` and `C ← 1000000`. It works for a bigger range but **still fails** for numbers outside that range.
- **Best: set both B and C to the first value input**, then check only the remaining nine values (so the counter test is `A < 9`). This works for **any** set of numbers.

KHALED

```
A ← 0
INPUT X
B ← X
C ← X
REPEAT
  INPUT X
  IF X > B
    THEN B ← X
  ENDIF
  IF X < C
    THEN C ← X
  ENDIF
  A ← A + 1
UNTIL A = 9
OUTPUT B, C
```

Worked example: find the errors. This algorithm should add up 10 positive numbers and output the total.

```
Counter ← 1
FOR Counter ← 1 TO 10
  REPEAT
    OUTPUT "Enter a positive whole number "
    INPUT Number
  UNTIL Number < 0
  Total ← Total + Counter
  Counter ← Counter + 1
  OUTPUT Total
NEXT Number
```

Error	Correction
`Total` is **never initialised**	Add `Total ← 0` **before** the loop
`Counter ← 1` before the FOR loop is not needed	Remove it. The FOR loop sets the counter
`UNTIL Number < 0` accepts only **negative** numbers	Change to `UNTIL Number > 0` (positive)
`Total ← Total + Counter` adds the **counter**	Change to `Total ← Total + Number`
`Counter ← Counter + 1` inside a FOR loop	Remove it. The FOR loop **adds 1 automatically**
`OUTPUT Total` is **inside the loop**	Move it **after** `NEXT` so it is output once
`NEXT Number` uses the **wrong variable**	Change to `NEXT Counter`

Corrected algorithm:

```
Total ← 0
FOR Counter ← 1 TO 10
  REPEAT
    OUTPUT "Enter a positive whole number "
    INPUT Number
  UNTIL Number > 0
  Total ← Total + Number
NEXT Counter
OUTPUT Total
```

Common types of error to look for:

- A variable that is **not initialised** (total, count, flag).
- The **wrong variable** used, or a **wrong loop end** (`NEXT` name, `UNTIL` / `WHILE` condition the wrong way round).
- A wrong **comparison** (< instead of >, < instead of <=).
- An **OUTPUT** in the wrong place (inside a loop instead of after it).
- Changing the loop counter inside a FOR loop.
- A **different variable name** used in the loop condition (for example `WHILE Number <> 0` when the variable is `Number1`).
- A wrong **start value** for maximum / minimum.

7.9 Writing and Amending Algorithms

Steps to follow:

1. **Understand the problem**: its purpose and the tasks to be done.
2. **Break it into sub-problems**: set-up, input, processing, storage (if needed), output.
3. Decide **what data is needed**, how it is **stored** (variables, arrays) and how results are **displayed**.
4. Draw a **structure diagram** if the problem is large.
5. Choose **flowchart or pseudocode** (follow the question if it tells you which).
6. **Write the algorithm** clearly: **meaningful names**, **indentation**, **precise conditions** (`Counter >= 10`, not “Counter ten or over”).
7. **Test it** with normal, abnormal, extreme and boundary data and a **trace table**.
8. **Correct any errors** and test again.

Example 1: concert tickets, $20 each, 10% discount for 10 or more tickets, 20% for 20 or more, maximum 25 tickets per transaction. Calculate the cost.

KHALED BIN AZIZ

```
REPEAT
  OUTPUT "How many tickets would you like to buy? "
  INPUT NumberOfTickets
UNTIL NumberOfTickets > 0 AND NumberOfTickets < 26
IF NumberOfTickets < 10
  THEN
    Discount ← 0
  ELSE
    IF NumberOfTickets < 20
      THEN
        Discount ← 0.1
      ELSE
        Discount ← 0.2
    ENDIF
ENDIF
Cost ← NumberOfTickets * 20 * (1 - Discount)
OUTPUT "Your tickets cost ", Cost
```

Example 2: input any number of positive numbers (stop when −1 is entered) and output the total and the average. A **WHILE** loop is used because the number of values is **not known** and the user might enter −1 straight away.

```
Total ← 0
Count ← 0
OUTPUT "Enter a positive number, -1 to finish "
INPUT Number
WHILE Number <> -1 DO
  Total ← Total + Number
  Count ← Count + 1
  OUTPUT "Enter a positive number, -1 to finish "
  INPUT Number
ENDWHILE
IF Count > 0
  THEN
    Average ← Total / Count
    OUTPUT "Total is ", Total
    OUTPUT "Average is ", Average
  ELSE
    OUTPUT "No numbers were entered"
ENDIF
```

Example 3 (nested loops): find the highest, lowest and average mark for each of four tests (Maths, Science, English, IT) for 600 students. An **outer FOR loop** runs once for each test and an **inner FOR loop** runs once for each student. The subject totals are **reset to their start values at the start of each outer loop**, while the overall totals are set once **before** the outer loop. To test it by dry run, reduce the number of students to 5 and the number of tests to 2.

> **Exam Tip:** To get full marks for writing an algorithm: **initialise** variables, use the **correct loop type**, **validate input** if asked, use **meaningful names**, show **every output**, and use **comments** (`//`) where it helps.

Model Answers

Name and describe the component parts of a computer system when it is decomposed.
- Inputs: the data entered while the system is running.
- Processes: the tasks done using the input data and any stored data.
- Outputs: information displayed or printed for the user.
- Storage: data saved in files for use in the future.

Explain what is meant by decomposition and top-down design.
- Decomposition breaks a complex problem into smaller parts that are easier to solve.
- Top-down design breaks a system into sub-systems, and each sub-system into smaller ones, until each does a single action. It lets several programmers work on different parts at the same time.

Explain the difference between validation and verification.
- Validation is an automatic check by the program that the data is reasonable (for example a range check) before it is accepted.
- Verification checks that the data has been copied accurately from the source (for example double entry or a visual check). Validation cannot tell if reasonable data is actually correct, so both are used.

A phone app splits a restaurant bill. It works for up to 12 diners and bills from $10 to $500. State the validation checks and give test data.
- Number of diners: range check (1 to 12), type check (integer), presence check.
- Bill: range check ($10 to $500), type check (a number), presence check.
- Normal data: 4 diners with a bill of $120 (each pays $30); 10 diners with a bill of $250 (each pays $25).
- Abnormal data: 0 or 13 diners, “six” diners, a bill of −20 or $600, a bill of “abc”.
- Extreme data: 1 and 12 diners; bills of $10 and $500 (accepted).
- Boundary data: diners 0 (rejected) and 1 (accepted); 12 (accepted) and 13 (rejected); bill $9.99 (rejected) and $10 (accepted); $500 (accepted) and $500.01 (rejected).

Describe how a bubble sort works.
- Each item is compared with the next item and they are swapped if they are in the wrong order.
- After each pass the last item compared is in its correct place, so the next pass is shorter.
- The passes repeat until a pass is completed with no swaps.

Describe how a linear search works.
- It starts at the first item and compares each item in turn with the value searched for.
- It stops when a match is found (position output) or after the last item (not found).

Explain why the programmer chose verification for the name, email address and password of a website user, and how it would be done.
- Verification makes sure the data entered is exactly what the user intended, because a typing mistake would still pass validation (for example a misspelt email address or password).
- Double entry: the user enters the email address and password twice and the program compares them. If they differ, an error message is shown and the user must enter them again. A screen check can also be used for the name.

State what a trace table is used for.
- It records the value of each variable (and any output) every time it changes during a dry run, so that the purpose of an algorithm can be found or errors can be located.

KHALED BIN AZIZ

Common Exam Questions

- Name the stages of the program development life cycle and describe what happens in each one.
- Explain abstraction and decomposition, with an example.
- Decompose a system into inputs, processes, outputs and storage.
- Draw a structure diagram for a system (for example a modulo 11 check digit program).
- State what a flowchart is and draw or complete one using the correct symbols.
- Explain the purpose of a given algorithm (pseudocode or flowchart) and state the output for given input.
- Write an algorithm in pseudocode or draw a flowchart using totalling, counting, average, maximum or minimum.
- Describe or write a linear search and a bubble sort.
- Name validation checks for given data and explain why each is suitable.
- Explain validation and verification, and describe double entry and a visual check.
- Give normal, abnormal, extreme and boundary test data with the expected results.
- Complete a trace table for a flowchart or pseudocode.
- Find and correct errors in a given algorithm, then rewrite it.
- Rewrite an algorithm using a different structure (for example IF statements into a CASE statement, or one loop into another).

Quick Revision

- **Life cycle:** analysis → design → coding → testing → maintenance.
- **Abstraction** = keep the key details, remove the rest. **Decomposition** = break a problem into smaller parts.
- **Computer system parts:** inputs, processes, outputs, storage. **Top-down design** uses stepwise refinement. **Structure diagram** = hierarchy chart.
- **Flowchart symbols:** terminator (START/STOP), process, input/output, decision (two labelled exits), flow lines.
- **Pseudocode:** keywords in CAPITALS, names start with a capital letter, indent by two spaces, ← for assignment.
- **FOR** = known repeats. **REPEAT … UNTIL** = runs at least once, stops when TRUE. **WHILE** = may never run, continues while TRUE.
- **Totalling** adds the value. **Counting** adds 1. **Initialise** both to 0.
- **Max/min:** set both to the first item. **Average** = total / count after the loop.
- **Linear search:** check each item in turn, use a `Found` flag. **Bubble sort:** swap neighbours that are out of order, repeat until a pass has no swaps.
- **Validation checks:** range, length, type, presence, format, check digit. It checks data is **reasonable**, not correct.
- **Verification:** double entry and screen / visual check. It checks data was **copied correctly**.
- **Test data:** normal (accepted), abnormal (rejected), extreme (largest and smallest accepted), boundary (edge pair: one accepted, one rejected).
- **Trace table:** one column per variable and one for the output. Record each change in a dry run.
- **Finding errors:** look for missing initialisation, wrong variable, wrong condition, output in the wrong place, wrong loop end.

KHALED BIN AZIZ